

Laborator 6 - Algoritmi

1. Scrieti și testati o funcție C care implementează algoritmul quicksort după pseudocodul prezentat la curs. Lucrați cu fișiere header.

<pre>MAIN() READ n //lungime array READ_ARRAY(a, n) QUICK_SORT(a, 1, n) PRINT_ARRAY(a, n) // sorted array END MAIN</pre>	<pre>QUICK_SORT(A, P, R) q := 0 IF P < R THEN q := PARTITION(A, P, R) QUICK_SORT(A, P, q - 1) QUICK_SORT(A, q + 1, R) END IF END QUICK_SORT</pre>
<pre>PARTITION(A, P, R) x := A[R] i := P - 1 FOR j := P TO R - 1 IF A[j] ≤ x THEN i := i + 1 //exchange A[i] with A[j] aux := A[i] A[i] := A[j] A[j] := aux END IF END FOR //exchange A[i+1] with A[R] A[R] := A[i + 1] A[i + 1] := x RETURN i + 1 END PARTITION</pre>	

2. Scrieți și testați o funcție C care implementează algoritmul heapsort după pseudocodul prezentat la curs. Lucrați cu fișiere header.

<pre> MAIN() HEAP_SIZE := 0; READ n //lungime array READ_ARRAY(a, n) HEAP_SORT(a, n) PRINT_ARRAY(a, n) // sorted array END MAIN </pre>	<pre> HEAP_SORT(A, N) BUILD_MAX_HEAP(A, N) FOR i := N DOWNT0 2 exchange A[1] with A[i] HEAP_SIZE := HEAP_SIZE -1 MAX_HEAPIFY(A, 1) END FOR END HEAP_SORT </pre>
<pre> BUILD_MAX_HEAP(A, N) HEAP_SIZE := N FOR i := [N/2] DOWNT0 1 MAX_HEAPIFY(A, i) END FOR END BUILD_MAX_HEAP LEFT (X) RETURN 2 * X END LEFT RIGHT (X) RETURN 2 * X + 1 END RIGHT </pre>	<pre> MAX_HEAPIFY (A, X) l := LEFT (X) r := RIGHT(X) IF l ≤ HEAP_SIZE AND A[l] > A[X] THEN largest := l ELSE largest := X END IF IF r ≤ HEAP_SIZE AND A[r] > A[largest] THEN largest := r END IF IF (largest != X) exchange A[X] with A[largest] MAX_HEAPIFY(A, largest) END IF END MAX_HEAPIFY </pre>

Obs: Atenție la modul în care se transmite variabila *HEAP_SIZE* în interiorul subprogramelor implementate in C!!!